FEU Institute of Technology
COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

Computer System and Architecture

Machine Problem

LABEXER3

Raymond C. Ricohermoso Jr.	Hadji Tejuco
Name of Student	Name of Professor
LABEXER3	03/11/2026
Data Performed	Date Submitted

List the steps that you perform to come up with the program.

Paste the source code and explain the functionalities and take a screenshot of the final output.

PROCESS: INPUT AND OUTPUT

```
@shinramyvnn ➜/workspaces/LABEXER3 (main) $ nasm -f elf32 labexer3.asm -o labexer3.o
@shinramyvnn ➜/workspaces/LABEXER3 (main) $ ld -m elf_i386 -e _start labexer3.o -o labexer3
@shinramyvnn ➜/workspaces/LABEXER3 (main) $ ./labexer3

=======================================
 COR -> Input, Output, Procedure, Macro
=======================================
1. Input -> Input the details (1st Part, 3rd Part)
2. Output -> Display the full info
3. Write -> Write the full details into a file
4. Read -> Reads the content of the file
5. Exit

Enter your choice: 1

--- ENTER STUDENT INFORMATION ---
Enrollment Status (e.g., ENROLLED):
ENROLLED
Student Number:
202511566
Full Name:
Raymond C. Ricohermoso Jr.
Phone Number:
09619186127
ZIP Code:
1870
College:
COMPUTER STUDIES
Program:
BSCSSE
Year Level:
2
Term (e.g., 1st term):
1st Term
School Year (e.g., SY 2026-2027):
SY 26-27
Complete Address:
B4 L9 GOLDEN PALM RESIDENCES

--- SUBJECTS AND UNITS ---
Enter 0 units if you are not taking a subject.
1. CCS0021 - INFORMATION MANAGEMENT (LEC)
 Units for this subject (whole number):
2
2. CCS0021L - INFORMATION MANAGEMENT (LAB)
 Units for this subject (whole number):
1
3. CS0001 - DISCRETE STRUCTURES I
 Units for this subject (whole number):
3
4. CS0003 - COMPUTER SYSTEMS & ARCHITECTURE
```

```
4. CS0003 - COMPUTER SYSTEMS & ARCHITECTURE
 Units for this subject (whole number):
2
5. CS0003L - COMPUTER SYSTEMS & ARCHITECTURE (LAB)
 Units for this subject (whole number):
1
6. CS0070 - OBJECT ORIENTED PROGRAMMING
 Units for this subject (whole number):
2
7. CS0070L - OBJECT ORIENTED PROGRAMMING-LAB
 Units for this subject (whole number):
1
8. GED0021 - SPECIALIZED ENGLISH PROGRAM 2
 Units for this subject (whole number):
3
9. GED0055 - MATHEMATICAL ANALYSIS
 Units for this subject (whole number):
3
10. GED0081 - COLLEGE PHYSICS 1 LECTURE
 Units for this subject (whole number):
2
11. GED0081L - COLLEGE PHYSICS 1 LABORATORY
 Units for this subject (whole number):
1

Student information and subject units recorded!

=======================================
 COR -> Input, Output, Procedure, Macro
=======================================
1. Input -> Input the details (1st Part, 3rd Part)
2. Output -> Display the full info
3. Write -> Write the full details into a file
4. Read -> Reads the content of the file
5. Exit

Enter your choice:
```

CHOICE 2:

```
=====================================
 COR -> Input, Output, Procedure, Macro
=====================================
1. Input -> Input the details (1st Part, 3rd Part)
2. Output -> Display the full info
3. Write -> Write the full details into a file
4. Read -> Reads the content of the file
5. Exit

Enter your choice: 2

=====================================
     SOLAR/COR TUITION CALCULATOR
=====================================


--- STUDENT INFORMATION SUMMARY ---
Enrollment Status: ENROLLED
Student Number: 202511566
Name: Raymond C. Ricohermoso Jr.
Phone: 09619186127
ZIP Code: 1870
College: COMPUTER STUDIES
Program: BSCSSE
Year Level: 2
Term: 1st Term
School Year: SY 26-27
Address: B4 L9 GOLDEN PALM RESIDENCES

--- SUBJECT UNIT SUMMARY ---
1. CCS0021 - INFORMATION MANAGEMENT (LEC) Units: 2
2. CCS0021L - INFORMATION MANAGEMENT (LAB) Units: 1
3. CS0001 - DISCRETE STRUCTURES I Units: 3
4. CS0003 - COMPUTER SYSTEMS & ARCHITECTURE Units: 2
5. CS0003L - COMPUTER SYSTEMS & ARCHITECTURE (LAB) Units: 1
6. CS0070 - OBJECT ORIENTED PROGRAMMING Units: 2
7. CS0070L - OBJECT ORIENTED PROGRAMMING-LAB Units: 1
8. GED0021 - SPECIALIZED ENGLISH PROGRAM 2 Units: 3
9. GED0055 - MATHEMATICAL ANALYSIS Units: 3
10. GED0081 - COLLEGE PHYSICS 1 LECTURE Units: 2
11. GED0081L - COLLEGE PHYSICS 1 LABORATORY Units: 1

--- TUITION FEE SUMMARY ---
Total units: 21
Unit price: PHP 1000
Tuition (total units x PHP 1,000): PHP 21000
Laboratory subjects taken: 4
Laboratory fee (PHP 2,500 each): PHP 10000
TOTAL ESTIMATED TUITION + LAB FEES: PHP 31000

Note: This estimate excludes miscellaneous, ID validation,
```

```
Note: This estimate excludes miscellaneous, ID validation,
and any other fees not specified in the calculation.

========================================
 COR -> Input, Output, Procedure, Macro
========================================
1. Input -> Input the details (1st Part, 3rd Part)
2. Output -> Display the full info
3. Write -> Write the full details into a file
4. Read -> Reads the content of the file
5. Exit

Enter your choice:
```

CHOICE 3

```
========================================
 COR -> Input, Output, Procedure, Macro
========================================
1. Input -> Input the details (1st Part, 3rd Part)
2. Output -> Display the full info
3. Write -> Write the full details into a file
4. Read -> Reads the content of the file
5. Exit

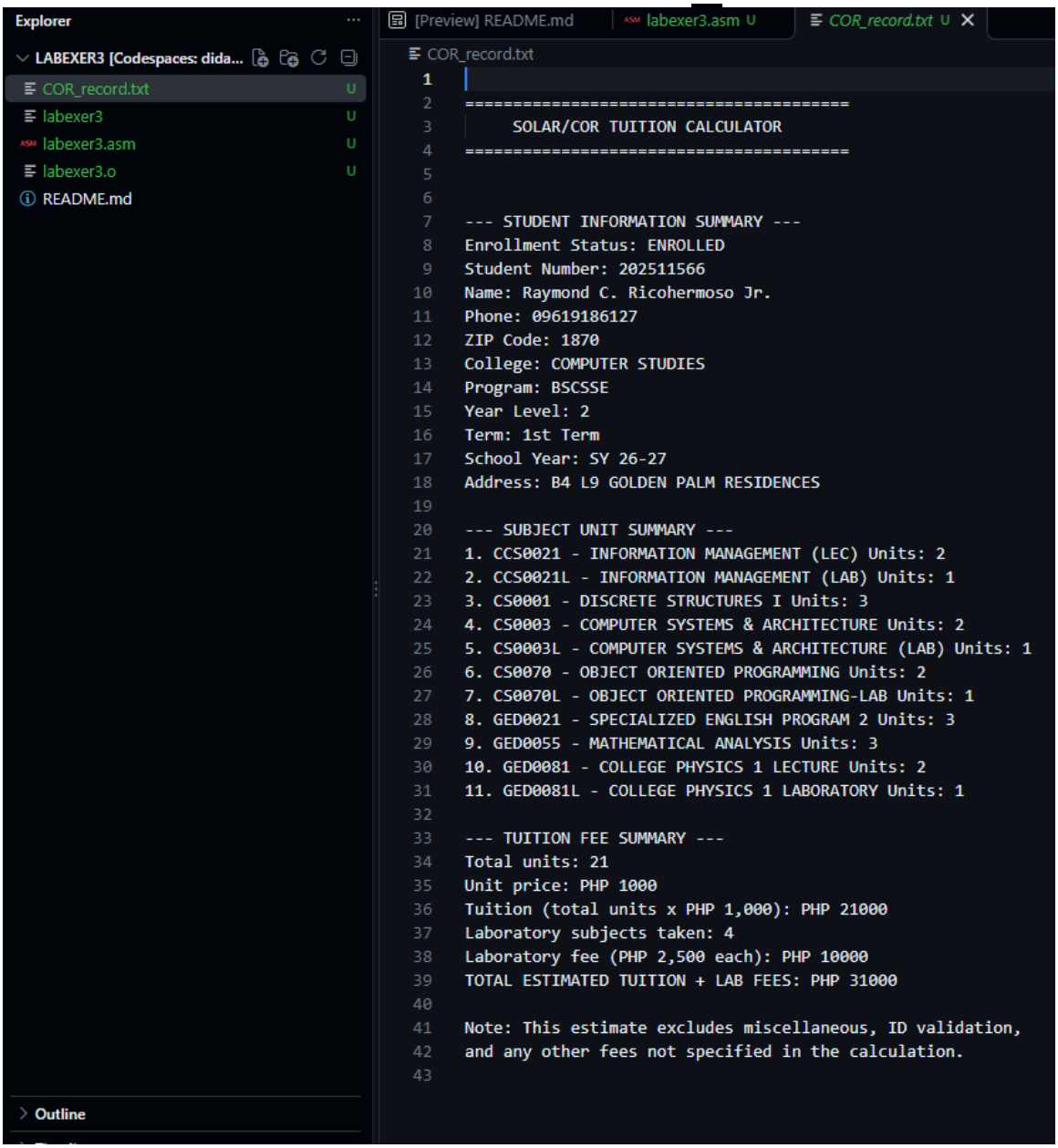
Enter your choice: 3

COR details saved to COR_record.txt

========================================
 COR -> Input, Output, Procedure, Macro
========================================
1. Input -> Input the details (1st Part, 3rd Part)
2. Output -> Display the full info
3. Write -> Write the full details into a file
4. Read -> Reads the content of the file
5. Exit

Enter your choice:
```

THE INFORMATION IS SAVED IN A TEXT FILE NAMED COR_record.txt



CHOICE 4

```
========================================
 COR -> Input, Output, Procedure, Macro
========================================
1. Input -> Input the details (1st Part, 3rd Part)
2. Output -> Display the full info
3. Write -> Write the full details into a file
4. Read -> Reads the content of the file
5. Exit

Enter your choice: 4

--- READING SAVED COR FILE ---

========================================
     SOLAR/COR TUITION CALCULATOR
========================================


--- STUDENT INFORMATION SUMMARY ---
Enrollment Status: ENROLLED
Student Number: 202511566
Name: Raymond C. Ricohermoso Jr.
Phone: 09619186127
ZIP Code: 1870
College: COMPUTER STUDIES
Program: BSCSSE
Year Level: 2
Term: 1st Term
School Year: SY 26-27
Address: B4 L9 GOLDEN PALM RESIDENCES

--- SUBJECT UNIT SUMMARY ---
1. CCS0021 - INFORMATION MANAGEMENT (LEC) Units: 2
2. CCS0021L - INFORMATION MANAGEMENT (LAB) Units: 1
3. CS0001 - DISCRETE STRUCTURES I Units: 3
4. CS0003 - COMPUTER SYSTEMS & ARCHITECTURE Units: 2
5. CS0003L - COMPUTER SYSTEMS & ARCHITECTURE (LAB) Units: 1
6. CS0070 - OBJECT ORIENTED PROGRAMMING Units: 2
7. CS0070L - OBJECT ORIENTED PROGRAMMING-LAB Units: 1
8. GED0021 - SPECIALIZED ENGLISH PROGRAM 2 Units: 3
9. GED0055 - MATHEMATICAL ANALYSIS Units: 3
10. GED0081 - COLLEGE PHYSICS 1 LECTURE Units: 2
11. GED0081L - COLLEGE PHYSICS 1 LABORATORY Units: 1

--- TUITION FEE SUMMARY ---
Total units: 21
Unit price: PHP 1000
Tuition (total units x PHP 1,000): PHP 21000
Laboratory subjects taken: 4
Laboratory fee (PHP 2,500 each): PHP 10000
```

```
TOTAL ESTIMATED TUITION + LAB FEES: PHP 31000

Note: This estimate excludes miscellaneous, ID validation,
and any other fees not specified in the calculation.

========================================
 COR -> Input, Output, Procedure, Macro
========================================
1. Input -> Input the details (1st Part, 3rd Part)
2. Output -> Display the full info
3. Write -> Write the full details into a file
4. Read -> Reads the content of the file
5. Exit

Enter your choice:
```

CHOICE 5: EXIT PROGRAM

```
========================================
 COR -> Input, Output, Procedure, Macro
========================================
1. Input -> Input the details (1st Part, 3rd Part)
2. Output -> Display the full info
3. Write -> Write the full details into a file
4. Read -> Reads the content of the file
5. Exit

Enter your choice: 5

Thank you for using the COR program!
@shinramyvnn →/workspaces/LABEXER3 (main) $
```

FULL SOURCE CODE:

```
GLOBAL _start

%macro PRINT 2
   pushad
   mov eax, 4
   mov ebx, [out_fd]
   mov ecx, %1
   mov edx, %2
   int 0x80
   popad
%endmacro

%macro ASK_STRING 4
   PRINT %1, %2
   push dword %4
```

```
    push dword %3
    call read_line
    add esp, 8
%endmacro

%macro SHOW_FIELD 3
    PRINT %1, %2
    push dword %3
    call print_string
    add esp, 4
    PRINT newline, newline_len
%endmacro

SECTION .data
    heading db 10, '========================================', 10
            db '     SOLAR/COR TUITION CALCULATOR     ', 10
            db '========================================', 10, 10
    heading_len equ $ - heading

    student_heading db 10, '--- ENTER STUDENT INFORMATION ---', 10
    student_heading_len equ $ - student_heading

    p_status        db 'Enrollment Status (e.g., ENROLLED):', 10
    p_status_len    equ $ - p_status
    p_student_no    db 'Student Number:', 10
    p_student_no_len equ $ - p_student_no
    p_name          db 'Full Name:', 10
    p_name_len      equ $ - p_name
    p_phone         db 'Phone Number:', 10
    p_phone_len     equ $ - p_phone
    p_zip           db 'ZIP Code:', 10
    p_zip_len       equ $ - p_zip
    p_college       db 'College:', 10
    p_college_len   equ $ - p_college
    p_program       db 'Program:', 10
    p_program_len   equ $ - p_program
    p_year_level    db 'Year Level:', 10
    p_year_level_len equ $ - p_year_level
    p_term          db 'Term (e.g., 1st term):', 10
    p_term_len      equ $ - p_term
    p_school_year   db 'School Year (e.g., SY 2026-2027):', 10
    p_school_year_len equ $ - p_school_year
    p_address       db 'Complete Address:', 10
    p_address_len   equ $ - p_address

    summary_heading db 10, '--- STUDENT INFORMATION SUMMARY ---', 10
    summary_heading_len equ $ - summary_heading

    l_status        db 'Enrollment Status: '
    l_status_len    equ $ - l_status
    l_student_no    db 'Student Number: '
    l_student_no_len equ $ - l_student_no
    l_name          db 'Name: '
    l_name_len      equ $ - l_name
    l_phone         db 'Phone: '
    l_phone_len     equ $ - l_phone
    l_zip           db 'ZIP Code: '
    l_zip_len       equ $ - l_zip
    l_college       db 'College: '
    l_college_len   equ $ - l_college
    l_program       db 'Program: '
    l_program_len   equ $ - l_program
    l_year_level    db 'Year Level: '
    l_year_level_len equ $ - l_year_level
```

```
l_term          db 'Term: '
l_term_len      equ $ - l_term
l_school_year   db 'School Year: '
l_school_year_len equ $ - l_school_year
l_address       db 'Address: '
l_address_len   equ $ - l_address

subjects_heading db 10, '--- SUBJECTS AND UNITS ---', 10
                 db 'Enter 0 units if you are not taking a subject.', 10
subjects_heading_len equ $ - subjects_heading

units_prompt     db ' Units for this subject (whole number): ', 10
units_prompt_len equ $ - units_prompt

unit_summary_heading db 10, '--- SUBJECT UNIT SUMMARY ---', 10
unit_summary_heading_len equ $ - unit_summary_heading

units_label      db ' Units: '
units_label_len  equ $ - units_label

fee_heading      db 10, '--- TUITION FEE SUMMARY ---', 10
fee_heading_len  equ $ - fee_heading

label_total_units   db 'Total units: '
label_total_units_len equ $ - label_total_units
label_unit_price    db 'Unit price: PHP '
label_unit_price_len equ $ - label_unit_price
label_tuition       db 'Tuition (total units x PHP 1,000): PHP '
label_tuition_len   equ $ - label_tuition
label_lab_subjects  db 'Laboratory subjects taken: '
label_lab_subjects_len equ $ - label_lab_subjects
label_lab_fee       db 'Laboratory fee (PHP 2,500 each): PHP '
label_lab_fee_len   equ $ - label_lab_fee
label_total_due     db 'TOTAL ESTIMATED TUITION + LAB FEES: PHP '
label_total_due_len equ $ - label_total_due

footer          db 10, 'Note: This estimate excludes miscellaneous, ID validation,', 10
                db 'and any other fees not specified in the calculation.', 10
footer_len      equ $ - footer

newline         db 10
newline_len     equ $ - newline
zero_text       db '0', 0

subj1           db '1. CCS0021 - INFORMATION MANAGEMENT (LEC)', 0
subj2           db '2. CCS0021L - INFORMATION MANAGEMENT (LAB)', 0
subj3           db '3. CS0001 - DISCRETE STRUCTURES I', 0
subj4           db '4. CS0003 - COMPUTER SYSTEMS & ARCHITECTURE', 0
subj5           db '5. CS0003L - COMPUTER SYSTEMS & ARCHITECTURE (LAB)', 0
subj6           db '6. CS0070 - OBJECT ORIENTED PROGRAMMING', 0
subj7           db '7. CS0070L - OBJECT ORIENTED PROGRAMMING-LAB', 0
subj8           db '8. GED0021 - SPECIALIZED ENGLISH PROGRAM 2', 0
subj9           db '9. GED0055 - MATHEMATICAL ANALYSIS', 0
subj10          db '10. GED0081 - COLLEGE PHYSICS 1 LECTURE', 0
subj11          db '11. GED0081L - COLLEGE PHYSICS 1 LABORATORY', 0

subject_table   dd subj1, subj2, subj3, subj4, subj5, subj6, subj7, subj8, subj9, subj10, subj11
lab_flags       db 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1

php_unit_price  equ 1000
php_lab_price   equ 2500

menu            db 10, '========================================', 10
                db ' COR -> Input, Output, Procedure, Macro ', 10
```

```
            db '========================================', 10
            db '1. Input -> Input the details (1st Part, 3rd Part)', 10
            db '2. Output -> Display the full info', 10
            db '3. Write -> Write the full details into a file', 10
            db '4. Read -> Reads the content of the file', 10
            db '5. Exit', 10, 10
            db 'Enter your choice: ', 0
    menu_len        equ $ - menu

    invalid_choice  db 10, 'Invalid choice. Please enter 1 to 5.', 10
    invalid_choice_len equ $ - invalid_choice

    invalid_units   db 10, 'Invalid units. Enter a whole number from 0 to 30:', 10
    invalid_units_len equ $ - invalid_units

    input_done      db 10, 'Student information and subject units recorded!', 10
    input_done_len  equ $ - input_done

    no_information  db 10, 'Please select Option 1 to enter COR information first.', 10
    no_information_len equ $ - no_information

    saved_notice    db 10, 'COR details saved to COR_record.txt', 10
    saved_notice_len equ $ - saved_notice

    save_failed     db 10, 'Error: unable to write COR_record.txt', 10
    save_failed_len equ $ - save_failed

    read_heading    db 10, '--- READING SAVED COR FILE ---', 10
    read_heading_len equ $ - read_heading

    read_failed     db 10, 'Error: COR_record.txt cannot be opened. Save it with Option 3 first.', 10
    read_failed_len equ $ - read_failed

    exit_message    db 10, 'Thank you for using the COR program!', 10
    exit_message_len equ $ - exit_message

    file_name       db 'COR_record.txt', 0

SECTION .bss
    status_buf        resb 128
    student_no_buf    resb 128
    name_buf          resb 128
    phone_buf         resb 128
    zip_buf           resb 128
    college_buf       resb 128
    program_buf       resb 128
    year_level_buf    resb 128
    term_buf          resb 128
    school_year_buf   resb 128
    address_buf       resb 128
    integer_buffer    resb 32
    units_array       resd 11
    total_units       resd 1
    lab_subject_count resd 1
    tuition_amount    resd 1
    lab_amount        resd 1
    total_amount      resd 1
    digit_buffer      resb 16
    out_fd            resd 1
    saved_fd          resd 1
    has_data          resb 1
    char_buf          resb 1
    file_buffer       resb 4096
```

```
SECTION .text
_start:
    mov dword [out_fd], 1
    mov byte [has_data], 0

.menu_loop:
    PRINT menu, menu_len
    call read_integer
    jnc .valid_choice
    cmp eax, -1
    je .exit
    jmp .invalid

.valid_choice:
    cmp eax, 1
    je .input
    cmp eax, 2
    je .output
    cmp eax, 3
    je .write
    cmp eax, 4
    je .read
    cmp eax, 5
    je .exit

.invalid:
    PRINT invalid_choice, invalid_choice_len
    jmp .menu_loop

.input:
    call input_details
    mov byte [has_data], 1
    PRINT input_done, input_done_len
    jmp .menu_loop

.output:
    cmp byte [has_data], 1
    jne .need_input
    call display_report
    jmp .menu_loop

.write:
    cmp byte [has_data], 1
    jne .need_input
    call save_report
    jmp .menu_loop

.read:
    call read_saved_file
    jmp .menu_loop

.need_input:
    PRINT no_information, no_information_len
    jmp .menu_loop

.exit:
    jmp quit_program

quit_program:
    PRINT exit_message, exit_message_len
    mov eax, 1
    xor ebx, ebx
    int 0x80
```

```
input_details:
    PRINT student_heading, student_heading_len
    ASK_STRING p_status, p_status_len, status_buf, 128
    ASK_STRING p_student_no, p_student_no_len, student_no_buf, 128
    ASK_STRING p_name, p_name_len, name_buf, 128
    ASK_STRING p_phone, p_phone_len, phone_buf, 128
    ASK_STRING p_zip, p_zip_len, zip_buf, 128
    ASK_STRING p_college, p_college_len, college_buf, 128
    ASK_STRING p_program, p_program_len, program_buf, 128
    ASK_STRING p_year_level, p_year_level_len, year_level_buf, 128
    ASK_STRING p_term, p_term_len, term_buf, 128
    ASK_STRING p_school_year, p_school_year_len, school_year_buf, 128
    ASK_STRING p_address, p_address_len, address_buf, 128

    mov dword [total_units], 0
    mov dword [lab_subject_count], 0
    PRINT subjects_heading, subjects_heading_len

    xor esi, esi
.subject_loop:
    cmp esi, 11
    jge .subjects_done

    mov eax, [subject_table + esi * 4]
    push eax
    call print_string
    add esp, 4
    PRINT newline, newline_len
    PRINT units_prompt, units_prompt_len

.get_units:
    call read_integer
    jnc .check_units
    cmp eax, -1
    je quit_program
    jmp .invalid_units

.check_units:
    cmp eax, 30
    ja .invalid_units
    mov [units_array + esi * 4], eax
    add [total_units], eax

    cmp eax, 0
    je .next_subject
    cmp byte [lab_flags + esi], 1
    jne .next_subject
    inc dword [lab_subject_count]

.next_subject:
    inc esi
    jmp .subject_loop

.invalid_units:
    PRINT invalid_units, invalid_units_len
    jmp .get_units

.subjects_done:
    mov eax, [total_units]
    imul eax, php_unit_price
    mov [tuition_amount], eax

    mov eax, [lab_subject_count]
    imul eax, php_lab_price
```

```
    mov [lab_amount], eax

    mov eax, [tuition_amount]
    add eax, [lab_amount]
    mov [total_amount], eax
    ret

display_report:
    PRINT heading, heading_len
    PRINT summary_heading, summary_heading_len

    SHOW_FIELD l_status, l_status_len, status_buf
    SHOW_FIELD l_student_no, l_student_no_len, student_no_buf
    SHOW_FIELD l_name, l_name_len, name_buf
    SHOW_FIELD l_phone, l_phone_len, phone_buf
    SHOW_FIELD l_zip, l_zip_len, zip_buf
    SHOW_FIELD l_college, l_college_len, college_buf
    SHOW_FIELD l_program, l_program_len, program_buf
    SHOW_FIELD l_year_level, l_year_level_len, year_level_buf
    SHOW_FIELD l_term, l_term_len, term_buf
    SHOW_FIELD l_school_year, l_school_year_len, school_year_buf
    SHOW_FIELD l_address, l_address_len, address_buf

    PRINT unit_summary_heading, unit_summary_heading_len
    xor esi, esi

.subject_loop:
    cmp esi, 11
    jge .fees

    mov eax, [subject_table + esi * 4]
    push eax
    call print_string
    add esp, 4

    PRINT units_label, units_label_len
    push dword [units_array + esi * 4]
    call print_uint
    add esp, 4
    PRINT newline, newline_len

    inc esi
    jmp .subject_loop

.fees:
    PRINT fee_heading, fee_heading_len

    PRINT label_total_units, label_total_units_len
    push dword [total_units]
    call print_uint
    add esp, 4
    PRINT newline, newline_len

    PRINT label_unit_price, label_unit_price_len
    push dword php_unit_price
    call print_uint
    add esp, 4
    PRINT newline, newline_len

    PRINT label_tuition, label_tuition_len
    push dword [tuition_amount]
    call print_uint
    add esp, 4
    PRINT newline, newline_len
```

```
    PRINT label_lab_subjects, label_lab_subjects_len
    push dword [lab_subject_count]
    call print_uint
    add esp, 4
    PRINT newline, newline_len

    PRINT label_lab_fee, label_lab_fee_len
    push dword [lab_amount]
    call print_uint
    add esp, 4
    PRINT newline, newline_len

    PRINT label_total_due, label_total_due_len
    push dword [total_amount]
    call print_uint
    add esp, 4
    PRINT newline, newline_len

    PRINT footer, footer_len
    ret

save_report:
    mov eax, 8
    mov ebx, file_name
    mov ecx, 420
    int 0x80
    test eax, eax
    js .fail
    mov [saved_fd], eax
    mov [out_fd], eax

    call display_report

    mov dword [out_fd], 1
    mov eax, 6
    mov ebx, [saved_fd]
    int 0x80

    PRINT saved_notice, saved_notice_len
    ret

.fail:
    PRINT save_failed, save_failed_len
    ret

read_saved_file:
    mov eax, 5
    mov ebx, file_name
    xor ecx, ecx
    xor edx, edx
    int 0x80
    test eax, eax
    js .fail
    mov [saved_fd], eax

    PRINT read_heading, read_heading_len

.read_loop:
    mov eax, 3
    mov ebx, [saved_fd]
    mov ecx, file_buffer
    mov edx, 4096
    int 0x80
```

```
    test eax, eax
    jle .close

    mov edi, eax
    xor esi, esi

.write_loop:
    mov eax, 4
    mov ebx, 1
    lea ecx, [file_buffer + esi]
    mov edx, edi
    sub edx, esi
    int 0x80
    test eax, eax
    jle .close
    add esi, eax
    cmp esi, edi
    jb .write_loop
    jmp .read_loop

.close:
    mov eax, 6
    mov ebx, [saved_fd]
    int 0x80
    ret

.fail:
    PRINT read_failed, read_failed_len
    ret

print_string:
    push ebp
    mov ebp, esp
    push ebx
    push esi
    mov esi, [ebp + 8]
    xor edx, edx
.count_chars:
    cmp byte [esi + edx], 0
    je .write_chars
    inc edx
    jmp .count_chars
.write_chars:
    test edx, edx
    jz .print_done
    mov eax, 4
    mov ebx, [out_fd]
    mov ecx, esi
    int 0x80
.print_done:
    pop esi
    pop ebx
    mov esp, ebp
    pop ebp
    ret

read_line:
    push ebp
    mov ebp, esp
    push ebx
    push esi
    push edi
    mov edi, [ebp + 8]
    mov esi, [ebp + 12]
```

```
    dec esi
    xor ebx, ebx
.read_next:
    push ebx
    mov eax, 3
    xor ebx, ebx
    mov ecx, char_buf
    mov edx, 1
    int 0x80
    pop ebx
    cmp eax, 0
    jg .got_char
    cmp ebx, 0
    jne .finished
    mov byte [edi], 0
    mov eax, -1
    jmp .done
.got_char:
    mov al, [char_buf]
    cmp al, 10
    je .finished
    cmp al, 13
    je .read_next
    cmp ebx, esi
    jae .read_next
    mov [edi + ebx], al
    inc ebx
    jmp .read_next
.finished:
    mov byte [edi + ebx], 0
    mov eax, ebx
.done:
    pop edi
    pop esi
    pop ebx
    mov esp, ebp
    pop ebp
    ret

read_integer:
    push ebp
    mov ebp, esp
    push ebx
    push esi
    push edx
    push dword 32
    push dword integer_buffer
    call read_line
    add esp, 8
    cmp eax, -1
    je .eof
    cmp eax, 0
    jle .invalid
    xor eax, eax
    mov esi, integer_buffer
.parse_digit:
    mov bl, [esi]
    test bl, bl
    jz .valid
    cmp bl, '0'
    jb .invalid
    cmp bl, '9'
    ja .invalid
    cmp eax, 99999
```

```
    ja .invalid
    imul eax, eax, 10
    sub bl, '0'
    movzx edx, bl
    add eax, edx
    inc esi
    jmp .parse_digit
.valid:
    clc
    jmp .done
.eof:
    mov eax, -1
    stc
    jmp .done
.invalid:
    xor eax, eax
    stc
.done:
    pop edx
    pop esi
    pop ebx
    mov esp, ebp
    pop ebp
    ret

print_uint:
    push ebp
    mov ebp, esp
    push ebx
    push esi
    push edi
    mov eax, [ebp + 8]
    test eax, eax
    jnz .convert_number
    push dword zero_text
    call print_string
    add esp, 4
    jmp .number_done
.convert_number:
    mov edi, digit_buffer
    add edi, 15
    mov byte [edi], 0
    mov ebx, 10
.convert_loop:
    xor edx, edx
    div ebx
    add dl, '0'
    dec edi
    mov [edi], dl
    test eax, eax
    jnz .convert_loop
    push edi
    call print_string
    add esp, 4
.number_done:
    pop edi
    pop esi
    pop ebx
    mov esp, ebp
    pop ebp
    ret
```